

Assignment 2: Theory & pthread

Omri Asudon 208853598 Matan Kronfeld 211465968

May 2026

1 Question 1: Parentheses

1.1 Pseudocode

Algorithm 1 Check Balanced Parentheses in Parallel

- 1: **Input:** Array S of characters of length n .
 - 2: **Output:** **true** if balanced, **false** otherwise.
 - 3: **Step 1:** In parallel for all $0 \leq i < n$:
 - 4: If $S[i] == '('$, set $A[i] \leftarrow 1$.
 - 5: Else if $S[i] == ')'$, set $A[i] \leftarrow -1$.
 - 6: Else, set $A[i] \leftarrow 0$.
 - 7: **Step 2:** Compute the parallel prefix sum of A into an array P .
 - 8: So $P[i] = \sum_{j=0}^i A[j]$.
 - 9: **Step 3:** Compute the minimum value in P into min_val using parallel reduction.
 - 10: **Step 4:** Check the last element $P[n-1]$ and min_val .
 - 11: If $P[n-1] == 0$ and $min_val \geq 0$, return **true**.
 - 12: Else, return **false**.
-

1.2 Explanation

We transform the problem of matching parentheses into a prefix sum problem. Every opening parenthesis '(' is assigned a weight of 1, every closing parenthesis ')' is assigned a weight of -1, and all other characters (numbers, operators) are assigned 0. A sequence of parentheses is balanced if and only if two conditions are met: 1. Every closing parenthesis has a corresponding preceding opening parenthesis. This means the prefix sum at any point must never drop below 0. 2. Every opening parenthesis has a corresponding closing parenthesis. This means the total sum at the end of the sequence must be exactly 0. We compute the prefix sums in parallel, then use a parallel reduction to find the minimum prefix sum, and finally verify both conditions.

1.3 Proof of Correctness

Proof. Let A be the weight array and P be its prefix sum array. ($\Rightarrow$) Assume S has balanced parentheses. By definition of balanced parentheses, there are an equal number of '(' and ')', which means $\sum_{i=0}^{n-1} A[i] = P[n-1] = 0$. Furthermore, in any valid prefix of a balanced sequence, the number of ')' cannot exceed the number of '('. Thus, $P[i] \geq 0$ for all i . Therefore, the minimum value in P is at least 0. The algorithm returns **true**.

($\Leftarrow$) Assume the algorithm returns **true**. Then $P[n-1] = 0$ and $\min_i P[i] \geq 0$. Since $P[n-1] = 0$, the total number of '(' equals the total number of ')'. Since $P[i] \geq 0$ for all i , no prefix has more ')' than '('. Thus, we can uniquely match every ')' with an available preceding '('. This satisfies the condition for proper nesting and balanced parentheses. $\square$

1.4 Runtime Analysis

Step 1 assigns values independently in $O(1)$ time with n processors. Step 2 calculates the prefix sum using the parallel scan algorithm, which takes $O(\log n)$ time with $O(n)$ work. Step 3 finds the minimum using parallel tree reduction, taking $O(\log n)$ time. Step 4 evaluates conditions in $O(1)$ time. Overall parallel time complexity: $O(\log n)$ using $O(n)$ processors.

2 Question 2: Counting subarrays with a given sum

2.1 Part 1: Positive numbers only

2.1.1 Pseudocode

Algorithm 2 Count Subarrays with sum b (Positive Array)

- 1: **Input:** Array A of n positive numbers, target sum b .
 - 2: **Step 1:** Compute prefix sum array P such that $P[i] = \sum_{j=1}^i A[j]$. Let $P[0] = 0$.
 - 3: **Step 2:** In parallel for all $1 \leq i \leq n$:
 - 4: Perform a binary search in P for the value $P[i] - b$.
 - 5: If found, set $C[i] \leftarrow 1$. Else, set $C[i] \leftarrow 0$.
 - 6: **Step 3:** Compute the total sum of array C using parallel reduction.
 - 7: **Step 4:** Return the total sum.
-

2.1.2 Explanation

Since all elements are strictly positive ($A[i] > 0$), the prefix sums array P is strictly monotonically increasing. A contiguous subarray $A[j \dots i]$ has a sum exactly equal to b if and only if $P[i] - P[j-1] = b$, which implies $P[j-1] =$

$P[i] - b$. Since P is sorted, we can simply binary search for the target value $P[i] - b$ in the array P .

2.1.3 Proof of Correctness

Proof. Because $A[i] > 0$ for all i , $P[i] > P[i-1]$, making P strictly monotonically increasing. For any fixed index i , the sum of the subarray ending at i and starting at j is $P[i] - P[j-1]$. We are looking for j such that $P[i] - P[j-1] = b \iff P[j-1] = P[i] - b$. Because P is strictly increasing, there is at most one index $j-1$ with this value. A binary search will correctly and uniquely identify if such a j exists. By doing this for all i independently and summing the results, we count all valid subarrays exactly once. $\square$

2.1.4 Runtime Analysis

Step 1: Parallel prefix sum takes $O(\log n)$ time. Step 2: Binary search takes $O(\log n)$ time. n processors perform this independently, so it takes $O(\log n)$ parallel time. Step 3: Parallel reduction sum takes $O(\log n)$ time. Overall parallel time complexity: $O(\log n)$ with $O(n)$ processors.

2.2 Part 2: General case

2.2.1 Pseudocode

Algorithm 3 Count Subarrays with sum b (General Array)

- 1: **Input:** Array A of n integers, target sum b .
 - 2: **Step 1:** Compute prefix sum array P such that $P[i] = \sum_{j=1}^i A[j]$. $P[0] = 0$.
 - 3: **Step 2:** Create an array of pairs $M[i] = (P[i], i)$ for $0 \leq i \leq n$.
 - 4: **Step 3:** Parallel Sort the array M primarily by the prefix sum value, and secondarily by index i .
 - 5: **Step 4:** In parallel for all $1 \leq i \leq n$:
 - 6: Let $target = P_{original}[i] - b$.
 - 7: Use binary search on M to find the range $[L, R]$ of pairs where the first element is $target$.
 - 8: Use binary search within the range $[L, R]$ (based on the second element) to count how many pairs have an index $j < i$.
 - 9: Let $C[i]$ be this count.
 - 10: **Step 5:** Return the sum of C using parallel reduction.
-

2.2.2 Explanation

When elements can be negative or zero, the prefix sum array is no longer monotonic. Therefore, multiple prefix sums can have the same value. A subarray $A[j \dots i]$ sums to b if $P[i] - P[j-1] = b$, so $P[j-1] = P[i] - b$ and $j-1 < i$. To find all such $j-1$ efficiently in parallel, we sort the prefix sums alongside

their original indices. For each index i , we binary search in the sorted array to find the block of identical prefix sums equal to $P[i] - b$. Within this block, the indices are sorted, so another binary search tells us exactly how many of them occurred before index i .

2.2.3 Proof of Correctness

Proof. For a subarray ending at i , its sum is b if the prefix sum before it equals $P[i] - b$. Since M contains all prefix sums paired with their indices and is sorted by the prefix sum value, all occurrences of the value $P[i] - b$ are contiguous in M . Let this contiguous block be $[L, R]$. We only care about occurrences that happen before i , i.e., index $j < i$. Since the secondary sorting key is the original index, the block $[L, R]$ is sorted by index. Thus, a binary search correctly counts the number of valid indices strictly less than i . Summing these counts over all i yields the exact number of valid subarrays. $\square$

2.2.4 Runtime Analysis

Step 1: Parallel prefix sum takes $O(\log n)$ time. Step 2: $O(1)$ time. Step 3: Parallel merge sort (e.g., Cole's algorithm) takes $O(\log n)$ time, or bitonic sort takes $O(\log^2 n)$ time. Step 4: Two binary searches take $O(\log n)$ time per processor, taking $O(\log n)$ parallel time overall. Step 5: Parallel reduction takes $O(\log n)$ time. Overall parallel time complexity: $O(\log^2 n)$ (or $O(\log n)$ depending on the chosen sorting algorithm) with $O(n)$ processors.

3 Question 3: k-th Ancestor in a Rooted Tree

3.1 Pseudocode

3.2 Explanation

This uses the pointer jumping (binary lifting) technique. The table $up[i][j]$ stores the 2^j -th ancestor of node i . The base case is $up[i][0] = parent[i]$. The recursive relation is that the 2^j -th ancestor is the 2^{j-1} -th ancestor of the 2^{j-1} -th ancestor. We compute this table in parallel layer by layer (for each j). Since the maximum jump distance is k , we only need $\log_2 k$ columns. After building the table, for each node i , we can find its k -th ancestor by iterating over the set bits in the binary representation of k and jumping accordingly.

3.3 Proof of Correctness

Proof. We prove by induction on j that $up[i][j]$ correctly stores the 2^j -th ancestor of i . Base case: $j = 0$, $up[i][0] = parent[i]$, which is the $2^0 = 1$ -st ancestor. Inductive step: Assume $up[x][j-1]$ is the 2^{j-1} -th ancestor of x . For any i , let $x = up[i][j-1]$ be its 2^{j-1} -th ancestor. The 2^j -th ancestor of i is exactly the 2^{j-1} -th ancestor of x , which is $up[x][j-1]$. Our algorithm sets

Algorithm 4 Parallel k -th Ancestor

```
1: Input: Parent array  $parent$  of size  $n$ ,  $k$ .
2: Step 1: Initialize an array  $up[n][\lfloor \log_2 k \rfloor + 1]$ .
3: Step 2: In parallel for all  $0 \leq i < n$ :
4:    $up[i][0] \leftarrow parent[i]$ 
5: Step 3: For  $j = 1$  to  $\lfloor \log_2 k \rfloor$ :
6:   In parallel for all  $0 \leq i < n$ :
7:     If  $up[i][j-1] \neq -1$ :
8:        $up[i][j] \leftarrow up[up[i][j-1]][j-1]$ 
9:     Else:
10:       $up[i][j] \leftarrow -1$ 
11: Step 4: In parallel for all  $0 \leq i < n$ :
12:   Let  $curr \leftarrow i$ .
13:   For  $j = 0$  to  $\lfloor \log_2 k \rfloor$ :
14:     If the  $j$ -th bit of  $k$  is 1:
15:        $curr \leftarrow up[curr][j]$ 
16:       If  $curr == -1$ , break
17:    $Ans[i] \leftarrow curr$ 
18: Return  $Ans$ 
```

$up[i][j] = up[x][j-1]$, thus the statement holds. For the query phase, k can be uniquely represented as a sum of powers of 2. Making jumps corresponding to these powers of 2 sequentially traces exactly k steps up the tree. Thus, $Ans[i]$ will correctly store the k -th ancestor. $\square$

3.4 Runtime Analysis

Step 1: Initialization takes $O(1)$ time. Step 2: Takes $O(1)$ time with n processors. Step 3: The loop runs $\log_2 k$ times. Inside the loop, the operations take $O(1)$ parallel time with n processors. Thus, it takes $O(\log k)$ time overall. Step 4: The inner loop runs $\log_2 k$ times, with each iteration taking $O(1)$. With n processors, this step takes $O(\log k)$ parallel time. Overall parallel time complexity: $O(\log k)$ with $O(n)$ processors.

4 Question 4: Trapping Rain Water

4.1 Pseudocode

4.2 Explanation

The amount of water that can be trapped above a specific bar i depends on the highest bar to its left and the highest bar to its right. Specifically, the trapped water is determined by the minimum of the maximum height to its left ($L[i]$) and the maximum height to its right ($R[i]$). The water above bar i is exactly $\min(L[i], R[i]) - height[i]$, bounded below by 0. We can use a parallel prefix

Algorithm 5 Parallel Trapping Rain Water

- 1: **Input:** Array $height$ of length n .
 - 2: **Step 1:** Compute parallel prefix maximum array L such that $L[i] = \max_{j=0}^i height[j]$.
 - 3: **Step 2:** Compute parallel suffix maximum array R such that $R[i] = \max_{j=i}^{n-1} height[j]$.
 - 4: **Step 3:** In parallel for all $0 \leq i < n$:
 - 5: $W[i] \leftarrow \max(0, \min(L[i], R[i]) - height[i])$
 - 6: **Step 4:** Return the sum of W using parallel reduction.
-

scan (using the ‘max’ operator instead of ‘+’) to compute the highest bar to the left for all positions in parallel. A similar suffix scan computes the highest bar to the right. Finally, we calculate the water at each index in $O(1)$ parallel time and sum the results.

4.3 Proof of Correctness

Proof. For any bar i , water can only be trapped if there are taller bars on both its left and its right. The water level at bar i will rise until it spills over the shorter of the two highest boundaries, which is $\min(\max_{j \leq i} height[j], \max_{k \geq i} height[k]) = \min(L[i], R[i])$. The actual volume of water trapped above bar i is the difference between this water level and the height of the bar itself, $\min(L[i], R[i]) - height[i]$. Since a bar cannot have negative water, we take $\max(0, \dots)$. Summing these amounts over all bars gives the total trapped water. The parallel prefix and suffix maximums compute $L[i]$ and $R[i]$ correctly by the associative property of the max operator. $\square$

4.4 Runtime Analysis

Step 1: Parallel prefix max takes $O(\log n)$ time using the parallel scan algorithm. Step 2: Parallel suffix max takes $O(\log n)$ time. Step 3: Calculating $W[i]$ independently takes $O(1)$ time with n processors. Step 4: Parallel sum reduction takes $O(\log n)$ time. Overall parallel time complexity: $O(\log n)$ with $O(n)$ processors.